

Unidad 6

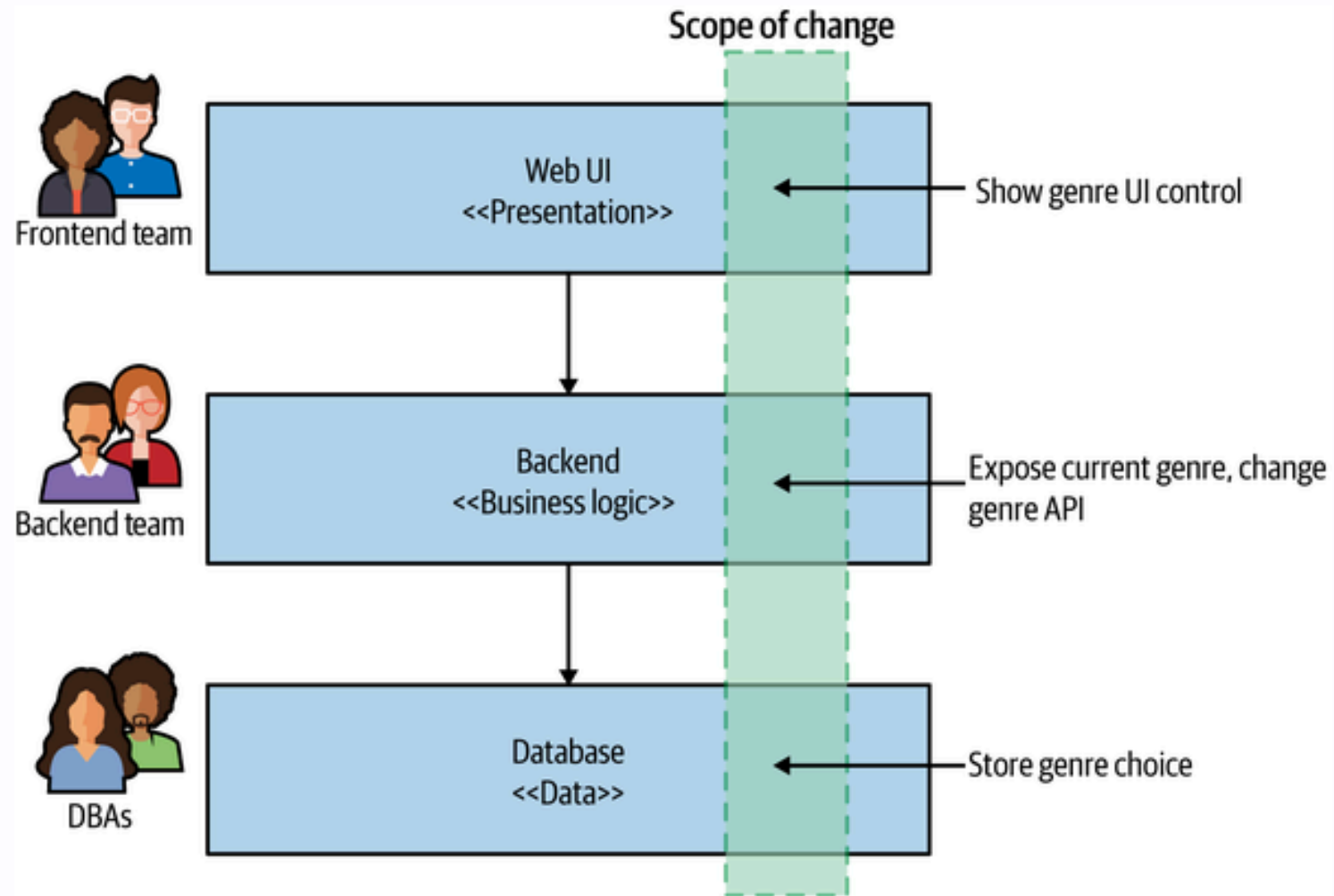
Interfaz

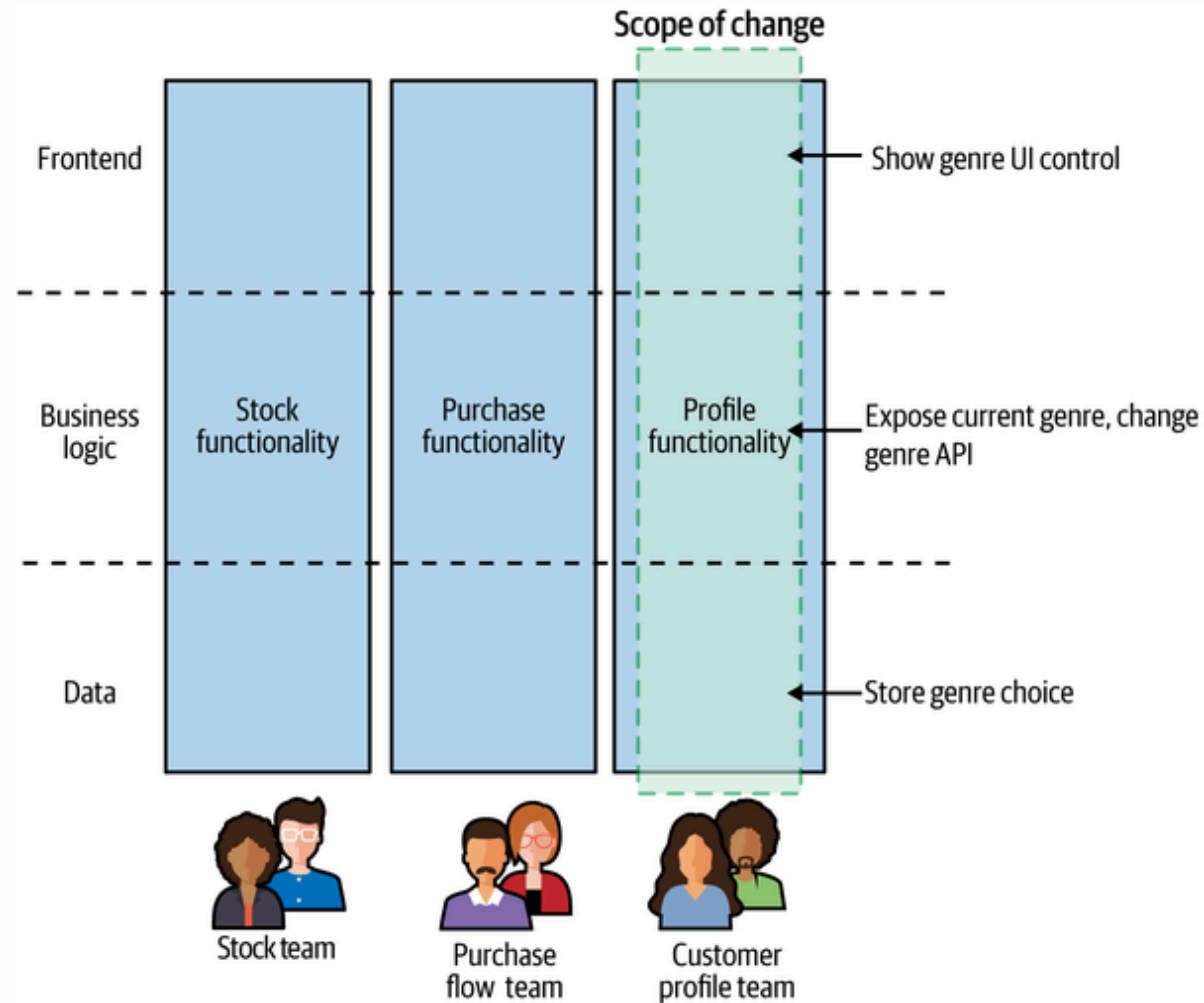
**Hasta ahora hemos desarrollado API
que, habitualmente, no son
utilizables por usuarios que esperan
(bellas) interfaces.**



Modelos de propiedad

En microservicios, habitualmente, cambia la estructura de responsabilidad/propiedad sobre el trabajo. Los equipos pasan de trabajar en capas de abstracción a hacerse cargo, de principio a fin, en funcionalidades.







Hacia equipos alineados

"Un equipo alineado se enfoca con un único y valioso flujo de trabajo... el equipo está facultado para construir y entregar valor del cliente o usuario tan rápido, de forma segura e independiente como sea posible, sin requerir transferencias a otros equipos para realizar partes del trabajo." Team topologies, de Skelton et al.

Full-stack teams por sobre *full-stack developers*. Un equipo responsable de principio a fin de entregar valor a los usuarios va a tener una mejor conexión/empatía con dichos usuarios.

Especialistas

Encontrar buenos especialistas (diseñadores, UX, front-end, etc) para cada equipo es un tema complejo. Existen dos modelos para lograr equipos *full-stack*:

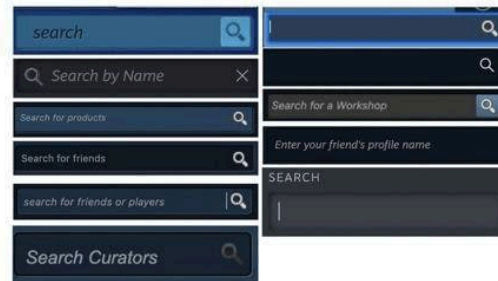
- Incluir un especialista por cada equipo.
- Tener un equipo (pequeño) de especialistas que ayude y enseñe a los equipos las habilidades que requieren.



Consistencia

Para asegurar consistencia en la interfaz es importante mantener ordenado un sistema de componentes y tener un sistema de diseño. Para eso existen frameworks (como [Chakra](#), Bootstrap o Material Design) y patrones como [Atomic Design](#).

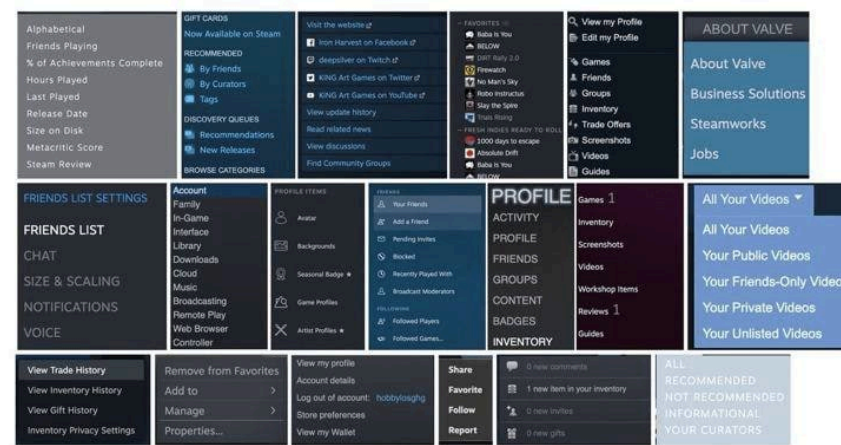
Search bars of Steam:



Tab menus of steam:



Drop-down/List menus of Steam:



Buttons of steam:

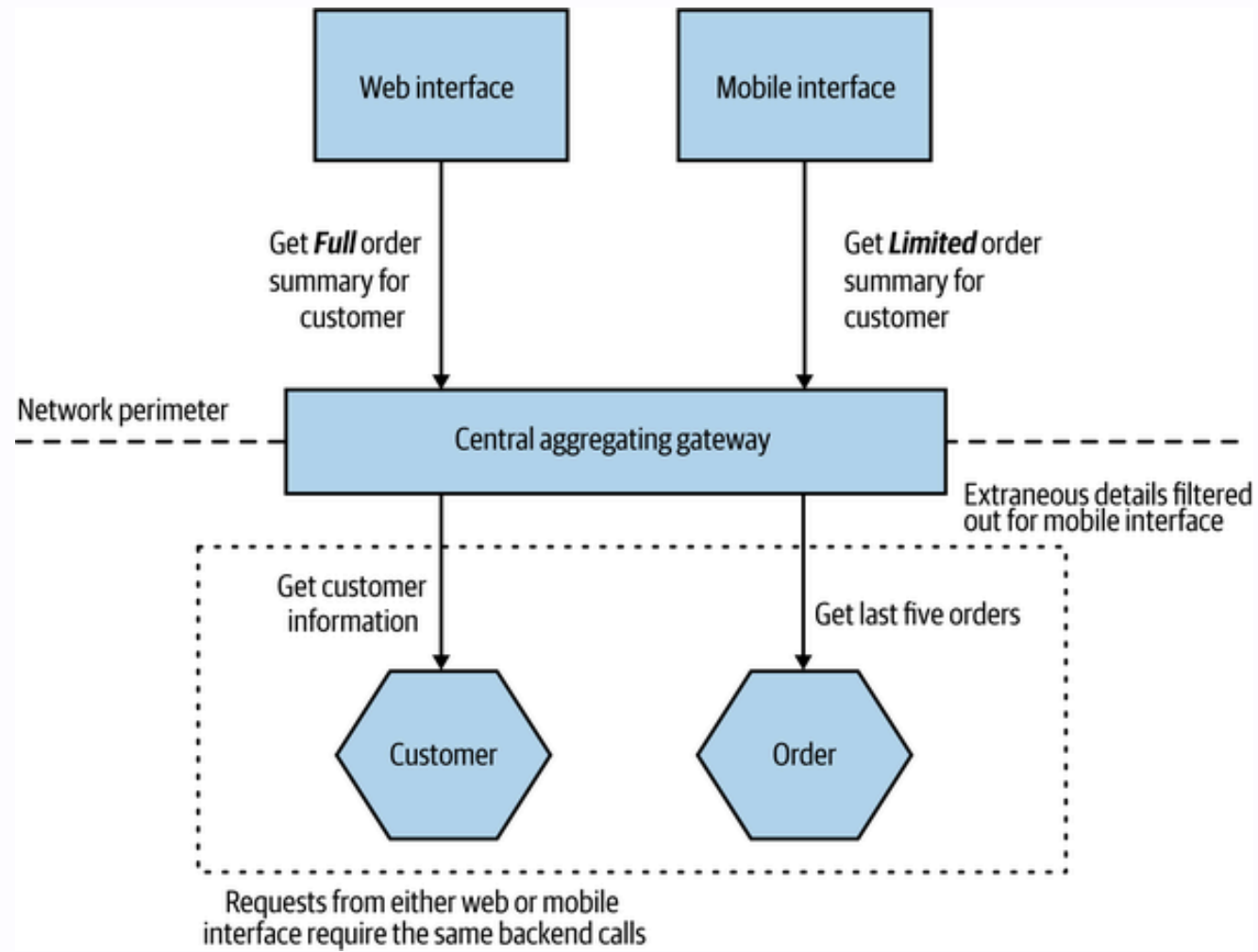


El no-sistema de diseño de Steam.



Gateway de agregación central

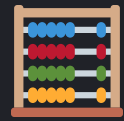
Se encuentra entre las interfaces de usuario y los microservicios. Realiza filtrado y agregación de las llamadas. Sin esta agregación, la interfaz tendría que realizar multiples llamadas (centenares en algunos casos) para obtener los datos de una vista y en dichas llamadas, se obtendrían datos no requeridos.





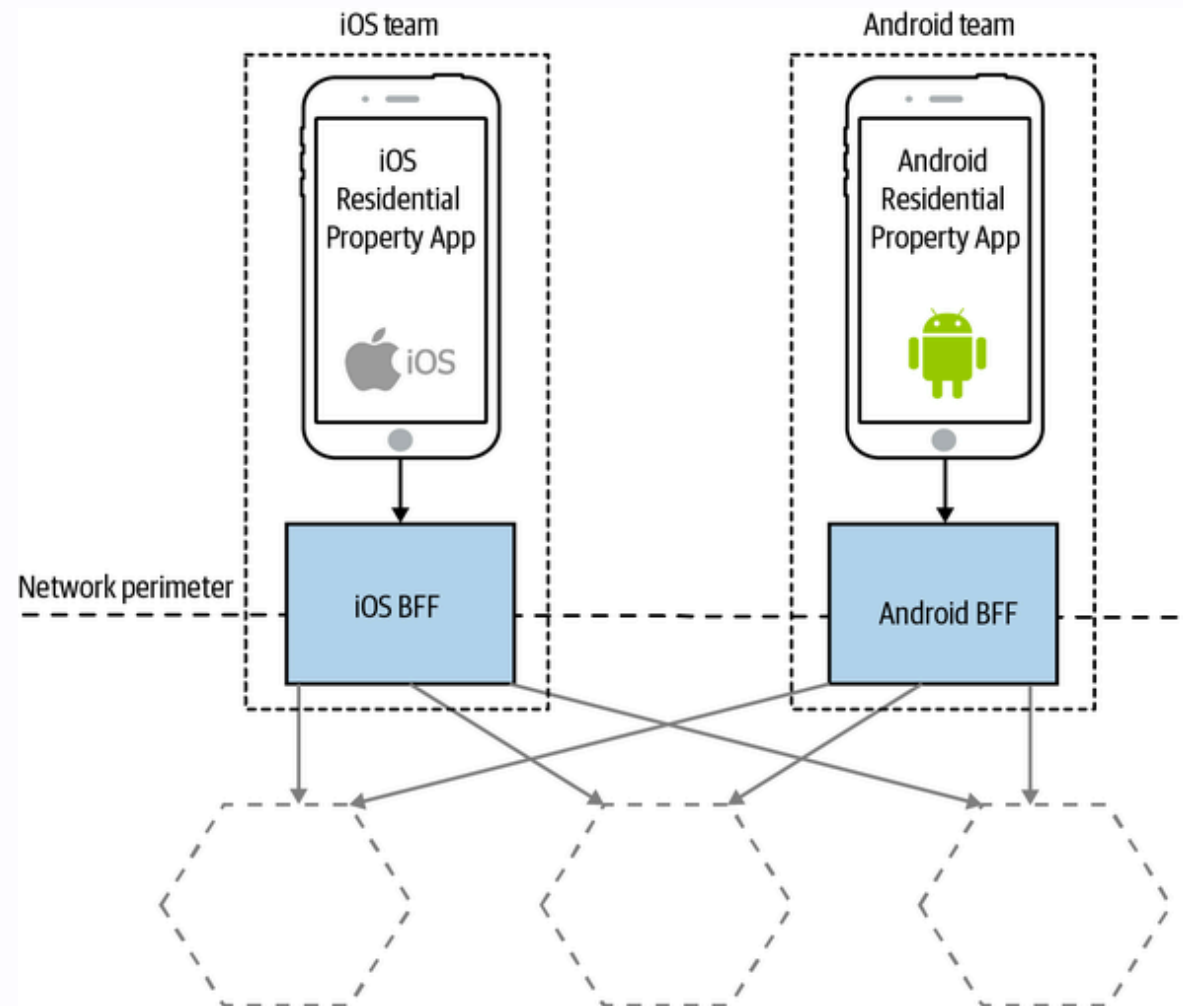
Back-end For Frontend (BFF)

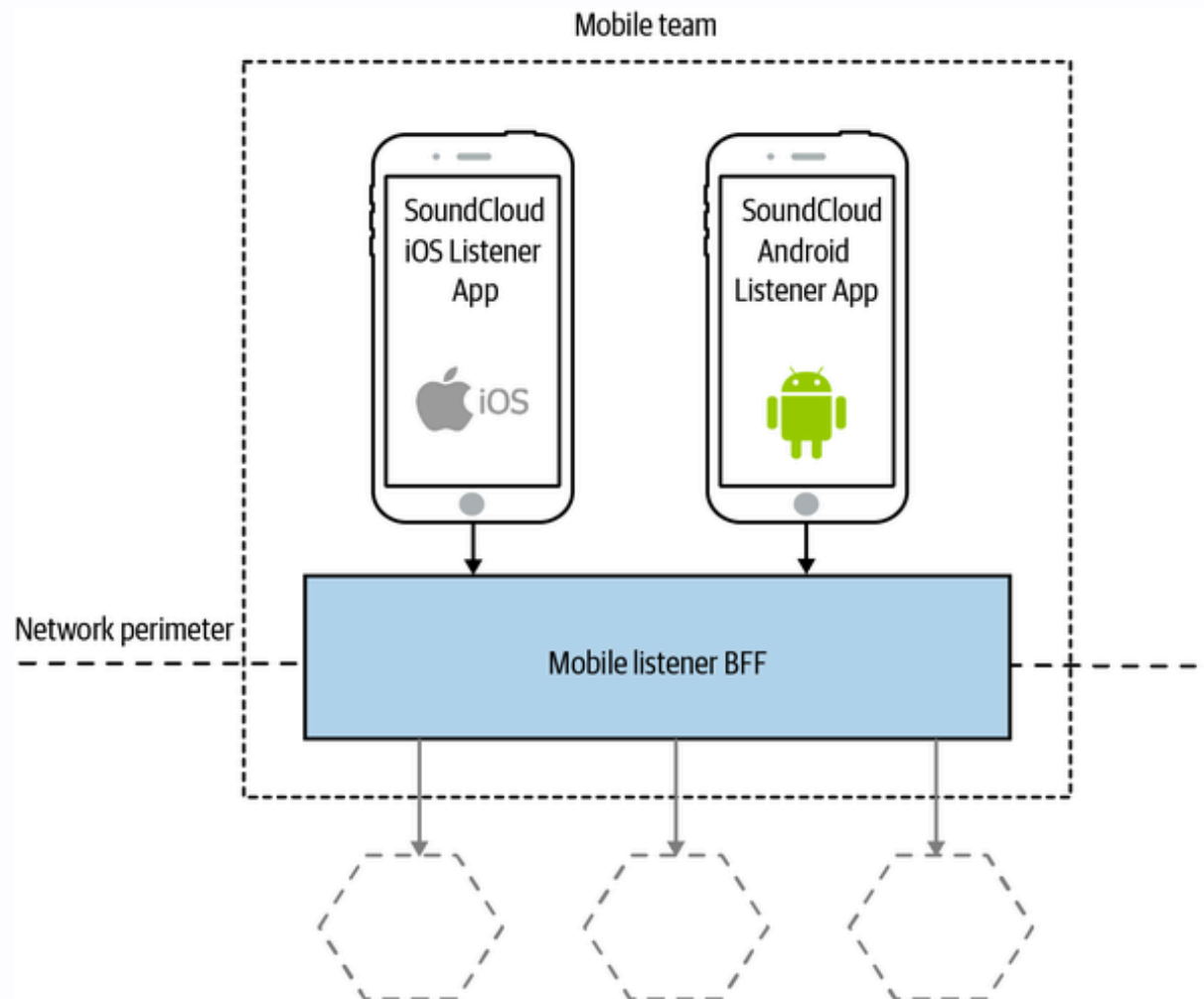
La principal distinción entre un BFF y un Gateway de agregación central es que un BFF tiene un solo propósito, es desarrollado para una interfaz (o un tipo de interfaz) de usuario. Este patrón demostró ser muy exitoso en ayudar a manejar las diferentes preocupaciones de las interfaces de usuario.



¿Cuantos BFF?

- Estrictamente un solo BFF para cada tipo diferente de cliente, aunque distintos clientes compartan el mismo tipo de interfaz.
- El mismo BFF para más de un tipo de cliente, pero para el mismo tipo de interfaz.







Micro-frontends

Dividir el gateway no basta si la interfaz sigue siendo un monolito. Un solo front-end que consume todos los servicios vuelve a acoplar a los equipos: cada cambio pasa por el mismo código, el mismo build y el mismo despliegue.

Composición por página

Cada equipo es dueño de páginas completas (por ejemplo, todo lo que cuelga de `/teams`). La navegación entre páginas une la aplicación. Es la forma más simple de dividir la interfaz y calza bien cuando la unidad natural del sitio es la página.

Composición por widget

Una misma página combina widgets desarrollados por distintos equipos. Requiere un contenedor común y cuidado extra con la consistencia visual y el peso de la página. Herramientas como Module Federation (Webpack) permiten cargar widgets desde builds separados.

¿Cuándo conviene?

- Cuando varios equipos necesitan desplegar su parte de la interfaz sin coordinarse con el resto.
- Si la interfaz es una SPA chica mantenida por un solo equipo, dividirla agrega complejidad sin beneficio.

Más detalles en Newman, capítulo 14.

Ejemplo: `./demo_06`

- Interfaz funcionando con React. Se conecta al *API Gateway*, en *GraphQL*, de `demo_05`.
- Usa **Chakra** como sistema de componentes/diseño.
- Si se cae el servicio de *teams* sigue funcionando la vista de jugadores.
- Se obtienen datos y se realizan mutaciones.



Tarea

Implementa una interfaz de usuario (web o móvil) para el sistema a desarrollar en este trabajo utilizando los microservicios del resto de los equipos. Para esto debes implementar en el servicio de *API Gateway* la conexión con los servicios y en la interfaz las funcionalidades obteniendo y modificando los datos desde API Gateway.

La network en *docker-compose* se va a llamar 'microsvcs'.

Incluye un video con una demostración de todas las funcionalidades implementadas. Sube el código a repositorios públicos.



Material complementario

- Building microservices: Designing fine-grained systems, Sam Newman (2021). O'Reilly. Capitulo 14.
- Skelton, M., Pais, M., & Malan, R. (2019). Team topologies: Organizing business and technology teams for fast flow. It Revolution.